



CS375 Spring 2026

Ralph Rostock

Week 1

Course Overview and Introduction to Software Engineering



Course Overview

- Course Number and Title: CS-375 – Software Engineering
- Credit Hours: 4
- Academic Term: Spring 2026
- Course Prerequisite: CS-371 Data Structures and Algorithms
- Class Meeting Location and Times: Thursday from 7:00 PM to 9:40 PM, PFA 107
- Office Hours
- Final Exam



ACM on Software Engineering

“...software engineering requires both the analytical and descriptive tools developed in computer science and the rigor that the engineering disciplines bring to the reliability and trustworthiness of the systems that software developers design and implement while working cohesively in a team environment.”





ACM on Software Engineering (continued)

Software Engineering:

- Must be viewed as a discipline with stronger ties to computer science than it has to other engineering fields
- Must share common characteristics with other engineering disciplines, including quantitative measurement, structured decision making, effective use of tools, and reusable software components
- Must apply engineering methods and practices to the development of software, with special emphasis on the development of large software systems
- Must integrate the principles of mathematics and computer science with engineering methodologies to create extensible software
- Must include secure code standards and quality control concepts of manufacturing process design
- Must emphasize communication skills, teamwork skills, and professional principles and best practices





IEEE definition of Software Engineering

“Software engineering may be defined as the systematic design and development of software products and the management of the software process”





Software Engineering is Not Programming

- Writing Code vs Building Systems
- Scale, teams, longevity and risk



Why Software Systems Fail

Software Fails because Humans Fail

- Unclear or changing requirements (building the wrong system)
- Poor system design or architectural decisions
- Inadequate testing and quality assurance
- Ignoring non-functional requirements (scalability, reliability, security)
- Human and process issues: communication, coordination, and handoffs
- Accumulated technical debt and lack of maintenance



Unclear or changing requirements (building the wrong system)

Denver International Airport Baggage Handling System (1990s)

- The automated baggage system requirements kept changing mid-development.
- The system was over-ambitious, poorly specified, and never stabilized.
- Result: years of delays, massive cost overruns, and eventual abandonment.

Lesson: If you don't know what you're building, code quality won't save you.



Poor system design or architecture decisions

Ariane 5 Flight V88 (1996)

- Reused Ariane 4 software without reconsidering system assumptions or specifically testing with newer rocket.
- A data conversion overflow caused both inertial reference systems to fail.
- Rocket self-destructed 37 seconds after launch (~\$370M loss).

Lesson: Correct code can work in one system but be architecturally wrong for a new or different system.



Inadequate testing and quality assurance

Knight Capital Trading Loss (2012)

- A dormant code path was accidentally activated in production.
- Inadequate testing and deployment checks failed to catch it.
- Result: \$440 million lost in ~45 minutes; company nearly collapsed.

Lesson: Untested paths are liabilities, not “inactive features.”



Ignoring non-functional requirements (scalability, reliability, security)

Healthcare.gov Launch (2013)

- System technically “worked” in small tests but collapsed under real traffic.
- Scalability, load handling, and resilience were afterthoughts.
- Result: site unusable at launch; emergency rebuild required.

Lesson: A system that works once is not a working system.



Human and process issues: communication, coordination, and handoffs

Mars Climate Orbiter (1999)

- One team used imperial units; another used metric units.
- No system-level checks caught the mismatch.
- Spacecraft burned up in Mars' atmosphere resulting in a loss of ~\$125M.

Lesson: Software systems fail at team boundaries, not just code boundaries.



Accumulated technical debt and lack of maintenance

Equifax Data Breach (2017)

- A known Apache Struts vulnerability was not patched.
- Outdated systems and poor maintenance practices left the system exposed.
- Result: personal data of ~147 million people compromised.

Lesson: Neglected software doesn't just degrade—it becomes dangerous.



Characteristics of Software Systems

- Complexity
- Changeability
- Invisibility
- Conformity



Complexity

- Software systems have many interacting components and states
- Small changes can have non-obvious ripple effects
- Complexity grows faster than system size (features $\times$ interactions)

Why it matters:

- Harder to understand, test, debug, and reason about than physical systems
- Primary driver of defects and failures



Changeability

- Software is expected to evolve continuously after deployment
- Requirements, environments, users, and regulations change
- “Finished” software is usually obsolete software

Why it matters:

- Design must anticipate change
- Maintainability often matters more than initial correctness



Invisibility

- Software has no physical form – no natural visual presentation
- Architecture, dependencies, and data flow are abstract
- Hard to “see” progress, structure, or decay

Why it matters:

- Makes estimation, communication, and management difficult
- Increases reliance on documentation, diagrams, and shared understanding



Conformity

- Software must conform to external constraints:
 - Hardware
 - Operating Systems
 - APIs
 - Laws, standards, organizational rules
- Many constraints are arbitrary and inconsistent

Why it matters:

- Complexity comes from the environment, not the problem itself
- Engineers spend more time adapting than inventing



The Software Engineering Life Cycle

Core Activities (Always Present)

Requirements -> Design -> Implementation -> Testing -> Deployment -> Maintenance

- Requirements: What problem are we solving, and for whom?
- Design: How will the system be structured to solve it?
- Implementation: Writing the code that realizes the design
- Testing: Gaining confidence that the system behaves as intended
- Deployment: Delivering the system to real users
- Maintenance: Fixing defects, adapting to change, and adapting the system

Every software system goes through all these activities, whether you plan for them or not



The Software Engineering Life Cycle (continued)

Lifecycle $\neq$ One-Time Sequence

Iterative vs Linear Views

- Linear view: Activities appear to happen once, in order
- Iterative view: Activities repeat and influence each other over time
- In practice, feedback flows backwards
 - Testing reveals design flaws
 - Deployment reveals missing requirements
 - Maintenance drives new implementation

The lifecycle describes what happens, not how you organize it



Roles on a Software Team

- Software Engineers
- Product Managers
- QA/Test Engineers
- UX/UI Designers
- DevOps/Site Reliability Engineers (SRE)

Software failures usually happen at the boundaries *between* roles



The Semester Project

- What students will build
- Constraints and expectations
- Team structure



How This Course Mirrors Industry

- Standups
- Reviews
- Retrospectives
- Incremental Delivery

**Team-based, long-running project with evolving requirements.
You are graded not just on what you build, but on how you work as a team.**



Standups

- Short, regular check-ins on progress and blockers
- Focus on communication, not status reporting
- Encourage shared ownership and early problem detection

Industry parallel: Daily or weekly team syncs



Reviews

- Teams demonstrate working software to others
- Feedback focuses on behavior, not just code
- Emphasizes learning to explain and defend design choices

Industry parallel: Sprint reviews, demos, stakeholder check-ins



Retrospectives

- Structured reflection on what worked and what didn't
- Focus on improving the process, not assigning blame
- Encourages continuous improvement over the semester

Industry parallel: Post-mortems and retrospectives



Incremental Delivery

- Software delivered in small, working increments
- Early versions may be incomplete—but must run
- Reduces risk and exposes problems sooner

Industry parallel: Iterative development and continuous delivery



Expectations and Professionalism

- Communication
- Collaboration
- Integrity and Accountability